Lab 07:

Question#01:

CODE:

```
#include <iostream>
using namespace std;

class Array2D
{
private:
    int **data;
    int rows, cols;

    // allocating memory
    void allocate()
    {
        data = new int *[rows];
        for (int i = 0; i < rows; i++)
            data[i] = new int[cols]{0}; // initialize to 0
    }

    // deallocating memory
    void deallocate()
    {
        for (int i = 0; i < rows; i++)
            delete[] data[i];
        delete[] data;
    }

public:
    Array2D() : rows(0), cols(0), data(nullptr) {}

    Array2D(int r, int c) : rows(r), cols(c)
    {
        allocate();
    }

    Array2D(const Array2D &other) : rows(other.rows), cols(other.cols)
    {
        allocate();
        for (int i = 0; i < rows; i++)
```

```
            for (int j = 0; j < cols; j++)
                data[i][j] = other.data[i][j];
    }

    // Destructor
    ~Array2D()
    {
        if (data)
            deallocate();
    }

    Array2D &operator=(const Array2D &other)
    {
        if (this == &other)
            return *this; // self-assignment guard

        if (data)
            deallocate();

        rows = other.rows;
        cols = other.cols;
        allocate();

        for (int i = 0; i < rows; i++)
            for (int j = 0; j < cols; j++)
                data[i][j] = other.data[i][j];

        return *this;
    }

    Array2D operator+(const Array2D &other) const
    {
        if (rows != other.rows || cols != other.cols)
        {
            cout << "Error: Size mismatch for +\n";
            return Array2D();
        }
        Array2D result(rows, cols);
        for (int i = 0; i < rows; i++)
            for (int j = 0; j < cols; j++)
                result.data[i][j] = data[i][j] + other.data[i][j];
        return result;
    }

    Array2D operator-(const Array2D &other) const
```

```
    {
        if (rows != other.rows || cols != other.cols)
        {
            cout << "Error: Size mismatch for -\n";
            return Array2D();
        }
        Array2D result(rows, cols);
        for (int i = 0; i < rows; i++)
            for (int j = 0; j < cols; j++)
                result.data[i][j] = data[i][j] - other.data[i][j];
        return result;
    }

    Array2D operator*(const Array2D &other) const
    {
        if (cols != other.rows)
        {
            cout << "Error: Invalid dimensions for *\n";
            return Array2D();
        }
        Array2D result(rows, other.cols);
        for (int i = 0; i < rows; i++)
            for (int j = 0; j < other.cols; j++)
                for (int k = 0; k < cols; k++)
                    result.data[i][j] += data[i][k] * other.data[k][j];
        return result;
    }

    int *operator[](int i)
    {
        return data[i];
    }

    const int *operator[](int i) const
    {
        return data[i];
    }

    void print() const
    {
        for (int i = 0; i < rows; i++)
        {
            for (int j = 0; j < cols; j++)
                cout << data[i][j] << "\t";
            cout << "\n";
```

```
        }
    }

    int getRows() const { return rows; }
    int getCols() const { return cols; }
};

int main()
{

    Array2D A(2, 3), B(2, 3);

    int val = 1;
    for (int i = 0; i < 2; i++)
        for (int j = 0; j < 3; j++)
            A[i][j] = val++;

    for (int i = 0; i < 2; i++)
        for (int j = 0; j < 3; j++)
            B[i][j] = val++;

    cout << "Matrix A:\n";
    A.print();
    cout << "\nMatrix B:\n";
    B.print();

    Array2D C = A + B;
    cout << "\nA + B:\n";
    C.print();

    Array2D D = A - B;
    cout << "\nA - B:\n";
    D.print();

    Array2D X(2, 3), Y(3, 2);
    X[0][0] = 1;
    X[0][1] = 2;
    X[0][2] = 3;
    X[1][0] = 4;
    X[1][1] = 5;
    X[1][2] = 6;

    Y[0][0] = 7;
    Y[0][1] = 8;
    Y[1][0] = 9;
```

```
    Y[1][1] = 10;
    Y[2][0] = 11;
    Y[2][1] = 12;

    Array2D Z = X * Y;
    cout << "\nX (2x3):\n";
    X.print();
    cout << "\nY (3x2):\n";
    Y.print();
    cout << "\nX * Y (2x2):\n";
    Z.print();

    Array2D E = A;
    cout << "\nCopy of A:\n";
    E.print();

    Array2D F;
    F = B;
    cout << "\nAssigned B to F:\n";
    F.print();

    return 0;
}
```

OUTPUT:

```
T-25276_Q1 } ; if ($?) { .\CT-25276_Q1 }
Matrix A:
1       2       3
4       5       6

Matrix B:
7       8       9
10      11      12

A + B:
8       10      12
14      16      18

A - B:
-6      -6      -6
-6      -6      -6

X (2x3):
1       2       3
4       5       6

Y (3x2):
7       8
9       10
11      12

X * Y (2x2):
58      64
139     154

Copy of A:
1       2       3
4       5       6

Assigned B to F:
7       8       9
10      11      12
PS D:\NED University\2nd Semester\Object Oriented Programming (OOPs)\Labs\Lab 07>
```

Question#02:

CODE:

```
#include <iostream>
```

Object Oriented Programming Lab 07:

```cpp
using namespace std;


class Array2D {
private:
    int** data;
    int rows, cols;

    void allocate() {
        data = new int*[rows];
        for (int i = 0; i < rows; i++)
            data[i] = new int[cols]{0};
    }

    void deallocate() {
        for (int i = 0; i < rows; i++)
            delete[] data[i];
        delete[] data;
    }

public:
    Array2D() : rows(0), cols(0), data(nullptr) {}

    Array2D(int r, int c) : rows(r), cols(c) { allocate(); }

    Array2D(const Array2D& other) : rows(other.rows), cols(other.cols) {
        allocate();
        for (int i = 0; i < rows; i++)
            for (int j = 0; j < cols; j++)
                data[i][j] = other.data[i][j];
    }

    ~Array2D() { if (data) deallocate(); }

    Array2D& operator=(const Array2D& other) {
        if (this == &other) return *this;
        if (data) deallocate();
        rows = other.rows; cols = other.cols;
        allocate();
        for (int i = 0; i < rows; i++)
            for (int j = 0; j < cols; j++)
                data[i][j] = other.data[i][j];
        return *this;
    }
```

```
    int* operator[](int i)             { return data[i]; }
    const int* operator[](int i) const { return data[i]; }

    int getRows() const { return rows; }
    int getCols() const { return cols; }

    void print() const {
        for (int i = 0; i < rows; i++) {
            for (int j = 0; j < cols; j++)
                cout << data[i][j] << "\t";
            cout << "\n";
        }
    }
};



bool searchMatrix(const Array2D& matrix, int target) {
    int rows = matrix.getRows();
    int cols = matrix.getCols();

    if (rows == 0 || cols == 0) return false;

    int lo = 0;
    int hi = rows * cols - 1;   // treat as flat array

    while (lo <= hi) {
        int mid = lo + (hi - lo) / 2;

        // Convert flat index → 2D index
        int midVal = matrix[mid / cols][mid % cols];

        if (midVal == target)
            return true;
        else if (midVal < target)
            lo = mid + 1;
        else
            hi = mid - 1;
    }

    return false;
}


int main() {
```

```
    // Build the matrix from the examples
    Array2D matrix(3, 4);
    int vals[3][4] = {
        { 1,  3,  5,  7},
        {10, 11, 16, 20},
        {23, 30, 34, 60}
    };
    for (int i = 0; i < 3; i++)
        for (int j = 0; j < 4; j++)
            matrix[i][j] = vals[i][j];

    cout << "Matrix:\n";
    matrix.print();
    cout << "\n";

    // Example 1: target = 3 → true
    int t1 = 3;
    cout << "Search " << t1 << ": "
         << (searchMatrix(matrix, t1) ? "true" : "false") << "\n";

    // Example 2: target = 13 → false
    int t2 = 13;
    cout << "Search " << t2 << ": "
         << (searchMatrix(matrix, t2) ? "true" : "false") << "\n";


    cout << "Search 1:  " << (searchMatrix(matrix,  1) ? "true" : "false") <<
"\n";
    cout << "Search 60: " << (searchMatrix(matrix, 60) ? "true" : "false") <<
"\n";
    cout << "Search 25: " << (searchMatrix(matrix, 25) ? "true" : "false") <<
"\n";

    return 0;
}
```

OUTPUT:

```
T-25276_Q2 } ; if ($?) { .\CT-25276_Q2 }
Matrix:
1       3       5       7
10      11      16      20
23      30      34      60

Search 3: true
Search 13: false
Search 1:  true
Search 60: true
Search 25: false
PS D:\NED University\2nd Semester\Object Oriented Programming (OOPs)\Labs\Lab 07>
```

Question#03:

CODE:

```cpp
#include <iostream>
#include <string>
using namespace std;

class Payroll; // knows Payroll exists.

class Employee
{
private:
    string name;
    int id;
    string designation;
    double salary;

public:
    Employee(string n, int i, string d, double s)
        : name(n), id(i), designation(d), salary(s) {}

    void display() const
    {
        cout << "ID: " << id
             << " | Name: " << name
             << " | Role: " << designation
             << " | Salary: " << salary << "\n";
    }
```

```
    friend class Payroll; // This makes Payroll a friend. Hence, all member
functions of Payroll can access Employee's private members.
};

class Payroll
{
public:
    void giveRaise(Employee &emp, double amount)
    {
        emp.salary += amount;
        cout << emp.name << "'s salary updated by +" << amount << "\n";
    }

    void givePercentRaise(Employee &emp, double percent)
    {
        double increase = emp.salary * (percent / 100.0);
        emp.salary += increase;
        cout << emp.name << "'s salary updated by " << percent << "%\n";
    }

    void promote(Employee &emp, string newDesignation, double newSalary)
    {
        emp.designation = newDesignation;
        emp.salary = newSalary;
        cout << emp.name << " promoted to " << newDesignation
             << " with salary " << newSalary << "\n";
    }
};

int main()
{

    // Create employees
    Employee e1("Usman Waqas", 101, "Junior Dev", 50000);
    Employee e2("Iman Khan", 102, "Designer", 45000);
    Employee e3("Faarid Amir", 103, "Manager", 80000);

    Payroll payroll;

    cout << "===== Before Update =====\n";
    e1.display();
    e2.display();
    e3.display();
```

```
    payroll.giveRaise(e1, 5000);

    payroll.givePercentRaise(e2, 10);

    payroll.promote(e3, "Senior Manager", 95000);

    cout << "\n===== After Update =====\n";
    e1.display();
    e2.display();
    e3.display();

    return 0;
}
```

OUTPUT:

```
===== Before Update =====
ID: 101 | Name: Usman Waqas | Role: Junior Dev | Salary: 50000
ID: 102 | Name: Iman Khan | Role: Designer | Salary: 45000
ID: 103 | Name: Faarid Amir | Role: Manager | Salary: 80000
Usman Waqas's salary updated by +5000
Iman Khan's salary updated by 10%
Faarid Amir promoted to Senior Manager with salary 95000

===== After Update =====
ID: 101 | Name: Usman Waqas | Role: Junior Dev | Salary: 55000
ID: 102 | Name: Iman Khan | Role: Designer | Salary: 49500
ID: 103 | Name: Faarid Amir | Role: Senior Manager | Salary: 95000
PS D:\NED University\2nd Semester\Object Oriented Programming (OOPs)\Labs\Lab 07>
```

Question#04:

CODE:

```
#include <iostream>
#include <string>
using namespace std;

class Employee
{
private:
```

```
    string name;
    int id;
    string designation;
    double salary;

public:
    Employee(string n, int i, string d, double s)
        : name(n), id(i), designation(d), salary(s) {}

    void display() const
    {
        cout << "ID: " << id
             << " | Name: " << name
             << " | Role: " << designation
             << " | Salary: " << salary << "\n";
    }

    friend void giveRaise(Employee &emp, double amount);
    friend void givePercentRaise(Employee &emp, double percent);
    friend void promote(Employee &emp, string newDesignation, double newSalary);
};

void giveRaise(Employee &emp, double amount)
{
    emp.salary += amount;
    cout << emp.name << "'s salary updated by +" << amount << "\n";
}

void givePercentRaise(Employee &emp, double percent)
{
    double increase = emp.salary * (percent / 100.0);
    emp.salary += increase;
    cout << emp.name << "'s salary updated by " << percent << "%\n";
}

void promote(Employee &emp, string newDesignation, double newSalary)
{
    emp.designation = newDesignation;
    emp.salary = newSalary;
    cout << emp.name << " promoted to " << newDesignation
         << " with salary " << newSalary << "\n";
}

int main()
{
```

```
    Employee e1("Usman Waqas", 101, "Junior Dev", 50000);
    Employee e2("Iman Khan", 102, "Designer", 45000);
    Employee e3("Faarid Amir", 103, "Manager", 80000);

    cout << "===== Before Update =====\n";
    e1.display();
    e2.display();
    e3.display();

    giveRaise(e1, 5000);
    givePercentRaise(e2, 10);
    promote(e3, "Senior Manager", 95000);

    cout << "\n===== After Update =====\n";
    e1.display();
    e2.display();
    e3.display();

    return 0;
}
```

OUTPUT:

```
===== Before Update =====
ID: 101 | Name: Usman Waqas | Role: Junior Dev | Salary: 50000
ID: 102 | Name: Iman Khan | Role: Designer | Salary: 45000
ID: 103 | Name: Faarid Amir | Role: Manager | Salary: 80000
Usman Waqas's salary updated by +5000
Iman Khan's salary updated by 10%
Faarid Amir promoted to Senior Manager with salary 95000

===== After Update =====
ID: 101 | Name: Usman Waqas | Role: Junior Dev | Salary: 55000
ID: 102 | Name: Iman Khan | Role: Designer | Salary: 49500
ID: 103 | Name: Faarid Amir | Role: Senior Manager | Salary: 95000
PS D:\NED University\2nd Semester\Object Oriented Programming (OOPs)\Labs\Lab 07>
```

Object Oriented Programming Lab 07:

Made By: Faarid Amir Shaikh CT-25276 :: Section: F